```
» [data-bridge-components~="button"]
» [data-controller~="bridge--button"] {
»   display: none !important;
» }
```

If you're wondering what in the world is going on here, well, you're not alone. Let's break it down line by line.

Remember when registering our bridge components added `bridge-components: ["button"]` to the user agent? The Hotwire Native Bridge uses this to add custom data attributes to the `<html>` element via JavaScript.

```
<html data-bridge-platform="ios" data-bridge-components="button">
```

The first line of new CSS, `[data-bridge-components~="button"]`, uses these data attributes on `<html>` to make sure the web view is equipped to handle a "button" component. The second line, `[data-controller~="bridge--button"]`, finds elements with `data-controller` set to "button". Together, these lines target the button element only for clients capable of handling this bridge component.

This means if the client does *not* register a "button" component, then the HTML button will remain visible. It only hides elements where a native button takes its place. This gives us a nice safety net when progressively upgrading components to native—older clients won't lose functionality; they will continue to use the pre-upgraded web interactions.

Now, before we go and make this component even better, let's take a second to reflect on how powerful this technique is. We can now add a *native* button to any page of our app *with only HTML*. Slap a `data-controller="bridge--button"` on something and boom—native button. And because the native interaction *actually clicks the web element*, we get all the behavior associated with it for free. Here, we're submitting a form, but we could execute some JavaScript, link to a new page, or anything else we can do on the web!